JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Package

java.lang.Object
    java.lang.Package

All Implemented Interfaces:

AnnotatedElement

```
public final class Package
extends Object
implements AnnotatedElement
```

Represents metadata about a run-time package associated with a class loader. Metadata includes annotations, versioning, and sealing.

Annotations for the run-time package are read from `package-info.class` at the same code source as classes in the run-time package.

The set of classes that make up the run-time package may implement a particular specification. The specification title, version, and vendor (indicating the owner/maintainer of the specification) can be provided when the `Package` is defined. An application can ask if the `Package` is compatible with a particular specification version by using the `Package.isCompatibleWith(String)` method. In addition, information about the actual classes that make up the run-time package can be provided when the `Package` is defined. This information consists of an implementation title, version, and vendor (indicating the supplier of the classes).

A `Package` may be explicitly defined with the `ClassLoader.definePackage(String, String, String, String, String, String, String, URL)` method. The caller supplies the specification and implementation titles, versions, and vendors. The caller also indicates whether the package is sealed. If a `Package` is not explicitly defined for a run-time package when a class in that run-time package is defined, then a `Package` is automatically defined by the class's defining class loader, as follows.

A `Package` automatically defined for classes in a named module has the following properties:

- The name of the package is derived from the binary names of the classes. Since classes in a named module must be in a named package, the derived name is never empty.
- The package is sealed with the module location as the code source, if known.
- The specification and implementation titles, versions, and vendors are unspecified.

- Any annotations on the package are read from `package-info.class` as specified above.

A `Package` automatically defined for classes in an unnamed module has the following properties:

- The name of the package is either `""` (for classes in an unnamed package) or derived from the binary names of the classes (for classes in a named package).
- The package is not sealed.
- The specification and implementation titles, versions, and vendors are unspecified.
- Any annotations on the package are read from `package-info.class` as specified above.

A `Package` can be obtained with the `Package.getPackage(String)` and `ClassLoader.getDefinedPackage(String)` methods. Every `Package` defined by a class loader can be obtained with the `Package.getPackages()` and `ClassLoader.getDefinedPackages()` methods.

Implementation Note:

The builtin class loaders do not explicitly define `Package` objects for packages in *named modules*. Instead those packages are automatically defined and have no specification and implementation versioning information.

See *Java Virtual Machine Specification*:

5.3 Creation and Loading

Since:

1.2

See Also:

The JAR File Specification: Package Sealing, `ClassLoader.definePackage(String, String, String, String, String, String, String, URL)`

Method Summary

All Methods	Static Methods	
Modifier and Type	**Method**	**Description**
`<A extends Annotation> A`	**`getAnnotation`**`(Class<A> annotationClas`	Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Annotation[]	**getAnnotations**()	Returns annotations that are *present* on this element.
<A extends Annotation> A[]	**getAnnotationsByType**(Class<A> annotationClas	Returns annotations that are *associated* with this element.
<A extends Annotation> A	**getDeclaredAnnotation**(Class<A> annotationClas	Returns this element's annotation for the specified type if such an annotation is *directly present*, else null.
Annotation[]	**getDeclaredAnnotations**()	Returns annotations that are *directly present* on this element.
<A extends Annotation> A[]	**getDeclaredAnnotationsBy**(Class<A> annotationClas	Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*.
String	**getImplementationTitle**()	Return the title of this package.
String	**getImplementationVendor**(	Returns the vendor that implemented this package, `null` is returned if it is not known.
String	**getImplementationVersion**	Return the version of this implementation.
String	**getName**()	Return the name of this package.
static Package	**getPackage**(String name)	**Deprecated.** If multiple class loaders delegate to each other and define classes with the same package name, and one such loader

		relies on the lookup behavior of `getPackage` to return a `Package` from a parent loader, then the properties exposed by the `Package` may not be as expected in the rest of the program.
static `Package[]`	**`getPackages`**()	Returns all of the `Packages` defined by the caller's class loader and its ancestors.
`String`	**`getSpecificationTitle`**()	Return the title of the specification that this package implements.
`String`	**`getSpecificationVendor`**()	Return the name of the organization, vendor, or company that owns and maintains the specification of the classes that implement this package.
`String`	**`getSpecificationVersion`**(	Returns the version number of the specification that this package implements.
`int`	**`hashCode`**()	Return the hash code computed from the package name.
`boolean`	**`isAnnotationPresent`**(`Class<? extends Annotation> annotationCl`	Returns true if an annotation for the specified type is *present*

		on this element, else false.
boolean	**isCompatibleWith**(String desired)	Compare this package's specification version with a desired version.
boolean	**isSealed**()	Returns true if this package is sealed.
boolean	**isSealed**(URL url)	Returns true if this package is sealed with respect to the specified code source `url`.
String	**toString**()	Returns the string representation of this Package.

Methods declared in class Object

clone, equals, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Details

getName

```
public String getName()
```

Return the name of this package.

Returns:

The fully-qualified name of this package as defined in section §6.5.3 of *The Java Language Specification*, for example, `java.lang`

getSpecificationTitle

```
public String getSpecificationTitle()
```

Return the title of the specification that this package implements.

Returns:

the specification title, `null` is returned if it is not known.

getSpecificationVersion

```
public String getSpecificationVersion()
```

Returns the version number of the specification that this package implements. This version string must be a sequence of non-negative decimal integers separated by "."'s and may have leading zeros. When version strings are compared the most significant numbers are compared.

Specification version numbers use a syntax that consists of non-negative decimal integers separated by periods ".", for example "2.0" or "1.2.3.4.5.6.7". This allows an extensible number to be used to represent major, minor, micro, etc. versions. The version specification is described by the following formal grammar:

SpecificationVersion:
Digits RefinedVersion$_{opt}$
RefinedVersion:
. *Digits*
. *Digits RefinedVersion*
Digits:
Digit
Digits
Digit:
any character for which `Character.isDigit(char)` returns `true`, e.g. 0, 1, 2, ...

Returns:

the specification version, `null` is returned if it is not known.

getSpecificationVendor

```
public String getSpecificationVendor()
```

Return the name of the organization, vendor, or company that owns and maintains the specification of the classes that implement this package.

Returns:

the specification vendor, `null` is returned if it is not known.

getImplementationTitle

```
public String getImplementationTitle()
```

Return the title of this package.

Returns:

the title of the implementation, `null` is returned if it is not known.

getImplementationVersion

```
public String getImplementationVersion()
```

Return the version of this implementation. It consists of any string assigned by the vendor of this implementation and does not have any particular syntax specified or expected by the Java runtime. It may be compared for equality with other package version strings used for this implementation by this vendor for this package.

Returns:

the version of the implementation, `null` is returned if it is not known.

getImplementationVendor

```
public String getImplementationVendor()
```

Returns the vendor that implemented this package, `null` is returned if it is not known.

Returns:

the vendor that implemented this package, `null` is returned if it is not known.

isSealed

```
public boolean isSealed()
```

Returns true if this package is sealed.

API Note:

Package sealing has no relationship with sealed classes or interfaces. Package sealing is specific to JAR files defined for classes in an unnamed module. See the `Package` class specification for details how a `Package` is defined as sealed package.

Returns:

true if the package is sealed, false otherwise

isSealed

```
public boolean isSealed(URL url)
```

Returns true if this package is sealed with respect to the specified code source `url`.

API Note:

Package sealing has no relationship with sealed classes or interfaces. Package sealing is specific to JAR files defined for classes in an unnamed

module. See the Package class specification for details how a Package is defined as sealed package.

Parameters:

`url` - the code source URL

Returns:

true if this package is sealed with respect to the given `url`

isCompatibleWith

```
public boolean isCompatibleWith(String desired)
                         throws NumberFormatException
```

Compare this package's specification version with a desired version. It returns true if this packages specification version number is greater than or equal to the desired version number.

Version numbers are compared by sequentially comparing corresponding components of the desired and specification strings. Each component is converted as a decimal integer and the values compared. If the specification value is greater than the desired value true is returned. If the value is less false is returned. If the values are equal the period is skipped and the next pair of components is compared.

Parameters:

`desired` - the version string of the desired version.

Returns:

true if this package's version number is greater than or equal to the desired version number

Throws:

`NumberFormatException` - if the current version is not known or the desired or current version is not of the correct dotted form.

getPackage

```
@Deprecated(since="9")
public static Package getPackage(String name)
```

Deprecated.
If multiple class loaders delegate to each other and define classes with the same package name, and one such loader relies on the lookup behavior of `getPackage` to return a `Package` from a parent loader, then the properties exposed by the `Package` may not be as expected in the rest of the program. For example, the `Package` will only expose annotations from the `package-info.class` file defined by the parent loader, even if annotations exist in a `package-info.class` file defined by a child loader. A more robust approach is to use the `ClassLoader.getDefinedPackage(String)` method which returns a `Package` for the specified class loader.

Finds a package by name in the caller's class loader and its ancestors.

If the caller's class loader defines a `Package` of the given name, the `Package` is returned. Otherwise, the ancestors of the caller's class loader are searched recursively (parent by parent) for a `Package` of the given name.

Calling this method is equivalent to calling `ClassLoader.getPackage(String)` on a `ClassLoader` instance which is the caller's class loader.

Parameters:

`name` - A package name, such as "`java.lang`".

Returns:

The `Package` of the given name defined by the caller's class loader or its ancestors, or `null` if not found.

Throws:

`NullPointerException` - if `name` is `null`.

See Also:

`ClassLoader.getDefinedPackage(String)`

getPackages

```
public static Package[] getPackages()
```

Returns all of the `Packages` defined by the caller's class loader and its ancestors. The returned array may contain more than one `Package` object of the same package name, each defined by a different class loader in the class loader hierarchy.

Calling this method is equivalent to calling `ClassLoader.getPackages()` on a `ClassLoader` instance which is the caller's class loader.

Returns:

The array of `Package` objects defined by this class loader and its ancestors

See Also:

ClassLoader.getDefinedPackages()

hashCode

```
public int hashCode()
```

Return the hash code computed from the package name.

Overrides:

hashCode in class Object

Returns:

the hash code computed from the package name.

See Also:

Object.equals(java.lang.Object),
System.identityHashCode(Object)

toString

```
public String toString()
```

Returns the string representation of this Package. Its value is the string "package " and the package name. If the package title is defined it is appended. If the package version is defined it is appended.

Overrides:

toString in class Object

Returns:

the string representation of the package.

getAnnotation

```
public <A extends Annotation> A getAnnotation
(Class<A> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *present,* else null.

Note that any annotation returned by this method is a declaration annotation.

Specified by:

getAnnotation in interface AnnotatedElement

Type Parameters:

A - the type of the annotation to query for and return if present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if present on this element, else null

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.5

isAnnotationPresent

```
public boolean isAnnotationPresent
(Class<? extends Annotation> annotationClass)
```

Returns true if an annotation for the specified type is *present* on this element, else false. This method is designed primarily for convenient access to marker annotations.

The truth value returned by this method is equivalent to:
`getAnnotation(annotationClass) != null`

Specified by:

`isAnnotationPresent` in interface `AnnotatedElement`

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

true if an annotation for the specified annotation type is present on this element, else false

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.5

getAnnotationsByType

```
public <A extends Annotation> A[] getAnnotationsByType
(Class<A> annotationClass)
```

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS §9.6), and if so, attempts to

find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

Specified by:

`getAnnotationsByType` in interface `AnnotatedElement`

Type Parameters:

`A` - the type of the annotation to query for and return if present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if associated with this element, else an array of length zero

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.8

getAnnotations

```
public Annotation[] getAnnotations()
```

Returns annotations that are *present* on this element. If there are no annotations *present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

Specified by:

`getAnnotations` in interface `AnnotatedElement`

Returns:

annotations present on this element

Since:

1.5

getDeclaredAnnotation

```
public <A extends Annotation> A getDeclaredAnnotation
(Class<A> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *directly present*, else null. This method ignores inherited annotations. (Returns null if no annotations are directly present on this element.)

Note that any annotation returned by this method is a declaration annotation.

Specified by:

`getDeclaredAnnotation` in interface `AnnotatedElement`

Type Parameters:

`A` - the type of the annotation to query for and return if directly present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if directly present on this element, else null

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.8

getDeclaredAnnotationsByType

```
public <A extends Annotation> A[] getDeclaredAnnotationsByType
(Class<A> annotationClass)
```

Description copied from interface: `AnnotatedElement`

Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*. This method ignores inherited annotations. If there are no specified annotations directly or indirectly present on this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getDeclaredAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS §9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation if one is present. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

getDeclaredAnnotationsByType in interface AnnotatedElement

Type Parameters:

A - the type of the annotation to query for and return if directly or indirectly present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if directly or indirectly present on this element, else an array of length zero

Throws:

NullPointerException - if the given annotation class is null

Since:

1.8

getDeclaredAnnotations

```
public Annotation[] getDeclaredAnnotations()
```

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

Specified by:

getDeclaredAnnotations in interface AnnotatedElement

Returns:

annotations directly present on this element

Since:

1.5

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.